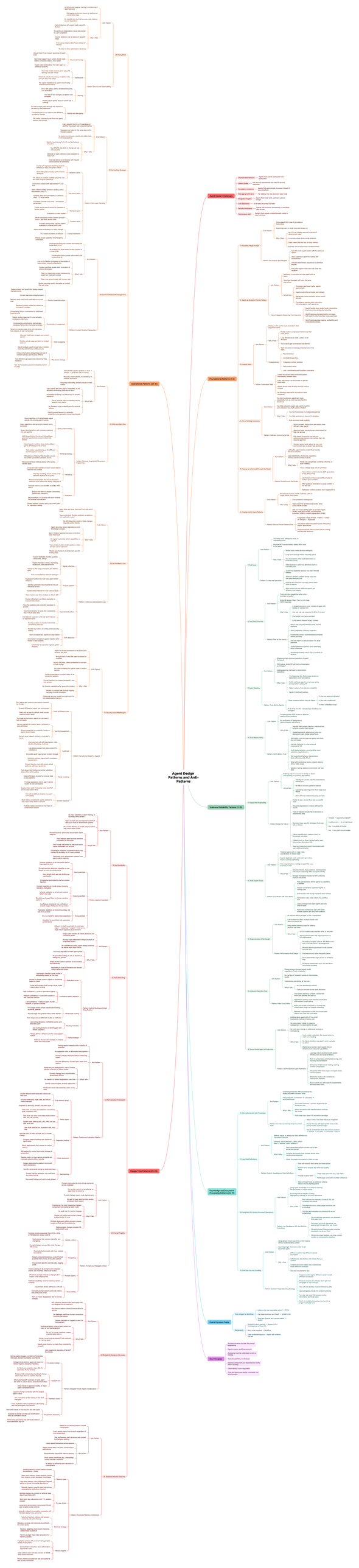

Agent Design Patterns and Anti-Patterns

Agent Design Challenges

- Unpredictable behavior
  - Agents that work in testing but fail in production
- Latency spikes
  - Sub-second requirements met with 30-second responses
- Compliance violations
  - Agents that approximate processes instead of executing them
- Debugging nightmares
  - No visibility into why decisions were made
- Integration fragility
  - Agents that break when upstream systems change
- Cost explosions
  - $0.10 tasks becoming $10 tasks
- Security blind spots
  - Agents with excessive permissions or unaudited data access
- Maintenance debt
  - Systems that require constant prompt tuning to stay functional

Foundational Patterns (1–6)

1. Monolithic Mega-Prompt

- Anti-Pattern
  - Giving agent 500+ lines of procedural instructions
  - Expecting exact, in-order execution every run
  - Why it fails
    - No LLM can reliably execute hundreds of deterministic steps
    - Long instructions dilute model attention
    - Steps toward the end rely on lossy memory
    - Business outcome becomes nondeterministic
- Pattern: Decompose and Delegate
  - Split into multi-agent system with focused sub-agents
  - Use a supervisor agent for routing and orchestration
  - Offload deterministic sequences to workflow engines
  - Keep each agent's instruction set small and targeted

2. Agent-as-Business-Process Fallacy

- Anti-Pattern
  - Replacing a controlled process graph with an agent
  - Assuming the agent will honor the same guarantees
  - Why it fails
    - Processes need exact paths; agents approximate
    - Agents lack enforced states and rollback
    - Replanning causes deviation where none is allowed
    - Compliance requires strict instruction-following agents can't guarantee
- Pattern: Separate Reasoning from Execution
  - Agents handle open-ended work: interpreting intent, resolving ambiguity, reasoning
  - Workflows execute deterministic processes with explicit state, branches, loops, approvals
  - Workflows guarantee logging, auditability, and controlled transitions

3. Invisible State

- Anti-Pattern
  - Relying on the LLM to "just remember" what happened earlier
  - Why it fails
    - Model creates compressed mental map that loses detail
    - Long threads cause older context to be dropped
    - Tool outputs get summarized and altered
    - State becomes increasingly distorted over more steps
  - Consequences
    - Repeated steps
    - Contradicting actions
    - Collapsing context windows
    - Hallucinated states
    - Lost commitments and forgotten constraints
- Pattern: Externalize State
  - Create structured state stored and passed intentionally between steps
  - Every step reads from and writes to specific state fields
  - Agents access state directly through tools or context
  - No inference required to reconstruct what happened

4. All-or-Nothing Autonomy

- Anti-Pattern
  - Too much autonomy: agent calls tools repeatedly, runs up cost, executes harmful sequences
  - Too little autonomy: agent asks user to confirm every decision, slow high-latency chatbot
  - Why it fails
    - Too much autonomy is unsafe and expensive
    - Too little autonomy is slow and frustrating
    - Both extremes break usability
- Pattern: Calibrate Autonomy by Risk
  - Action budgets: limit actions per session (max API calls, max spend)
  - Approval gates: require human confirmation for high-risk actions
  - Risk-based thresholds: low-risk runs autonomously, medium-risk notifies, high-risk requires approval
  - Context-aware levels: adjust by user role, environment (dev vs prod), task sensitivity

5. Passing As-Is Content Through the Model

- Anti-Pattern
  - Letting the agent rewrite content that must be delivered verbatim
  - Legal statements, disclosures, regulatory notices, precise numerical values
  - Why it fails
    - Model may paraphrase, condense, reformat, or alter meaning
    - This is a design issue, not an LLM issue
- Pattern: Route Around the Model
  - Tools deliver content directly (PDF generation, email, SMS, portal)
  - Use Human-in-the-Loop workflow nodes to surface exact content
  - MCP audience annotation to signal content is for end user
  - Reference content location, don't regenerate it

6. Chasing Exotic Agent Patterns

- Anti-Pattern
  - Reaching for Swarm, CUGA, CodeAct, LLM-as-Judge before fixing basics
  - Why it fails
    - Core problem is misdiagnosis
    - Teams reach for architectural novelty when actual failure is basic
    - Task too broad (AP#1), agent as process engine (AP#2), state drift (AP#3), unconstrained autonomy (AP#4), content regeneration (AP#5)
- Pattern: Exhaust Proven Patterns First
  - Progression: Single Prompt → ReAct → Chain-of-Thought → Supervisor
  - Only adopt advanced patterns after exhausting simpler approaches
  - Diagnose specific failure modes before making architectural decisions

Scale and Reliability Patterns (7–15)

7. Tool Soup

- Anti-Pattern
  - Too many tools, ambiguous tools, or overlapping tools
  - Multiple MCP servers blindly adding 100+ tools to the agent
  - Why it fails
    - Similar tools create decision ambiguity
    - Large tool catalogs inflate reasoning space
    - Tool descriptions often lack determinism or parameter clarity
    - Token explosion: same tool definitions sent to LLM every turn
- Pattern: Curate and Specialize
  - Cluster by capability: expose only task-relevant subsets
  - Abstract variants: combine similar tools into one parameterized tool
  - Explicit MCP selection: manually select which tools to expose
  - Role-based tool sets: different agents get different tool subsets

8. Tool Data Overload

- Anti-Pattern
  - Tools returning megabytes when only a summary is needed
  - Entire DB dumps, binary files to LLM, huge JSON responses
  - Why it fails
    - U-shaped accuracy curve: models struggle with middle-of-context info
    - One tool call can consume 50–80% of context
    - Cost spikes from large payloads
    - LLMs cannot interpret binary formats
- Pattern: Filter at the Source
  - Return only required fields/records, not full payloads
  - Apply pagination, filtering, projection
  - For binaries: extract text/metadata/summaries before returning
  - Use sub-Agent as data processor for large payloads
  - Artifact/Reference pattern: store externally, return reference
  - Streaming/chunking: return TOCs, excerpts, or sections

9. Agent Washing

- Anti-Pattern
  - Wrapping single-purpose operations in agent framework
  - FAQ lookup, single API call, text summarization as "AI Agent"
  - Adding planning overhead to deterministic operations
  - Why it fails
    - The Reasoning Tax: ReAct loops double or triple token count and latency
    - Prefill overhead: agent must process entire prompt before calling API
    - Higher variance from decision instability
    - Harder to test and maintain
- Pattern: Tools Before Agents
  - Three questions before using an Agent
    - Is the tool selection dynamic?
    - Is the path conditional?
    - Is there a feedback loop?
  - If all three are "No" it should be a Tool/Flow, not an Agent

10. Trust Before Verify

- Anti-Pattern
  - Adopting public MCP servers or external agents without auditing
  - No verification of idempotency, destructiveness, side effects
  - Why it fails
    - Security risks: prompt injection, malicious tool behavior, supply chain attacks
    - Operational issues: destructive tools, non-idempotent calls, hidden side effects
- Pattern: Verify Before Trust
  - Add safety controls: approval gates, rate limits, dry-run modes
  - Maintain fallbacks for when external components fail
  - Audit implementation: error handling, input validation, logs/telemetry
  - Test operational behavior: idempotency, destructiveness, side effects
  - Wrap with monitoring: inputs, outputs, latency, errors, token usage
  - Sandbox testing: isolated environment with test data

11. Happy Path Engineering

- Anti-Pattern
  - Building only for success: no retries, no graceful degradation
  - Why it fails
    - Training on success only: failure scenarios filtered out
    - No failure recovery patterns learned
    - Cascading reasoning errors from single tool failure
    - Silent failures swallowed by long prompts
- Pattern: Design for Failure
  - Partial re-plan: recover from last successful state
  - Graceful degradation: continue with partial results
  - Train on failures: include failure scenarios in eval/training data
  - Recovery trees: specific strategies for known failure classes
    - Timeout → exponential backoff
    - Invalid params → re-prompt/repair
    - 4xx → escalate to human
    - 5xx → retry with circuit breaker
  - Failure classification: transient (retry) vs permanent (escalate)
  - Fallback tools or flows: cached paths, read-only modes, alternative APIs
  - Dead-end detection: explicit termination with user-visible summaries

12. Multi-Agent Chaos

- Anti-Pattern
  - Multiple agents with no clear roles, coordination, or shared state
  - Agents duplicate work, contradict each other, bounce control indefinitely
  - Over-segmentation: creating an agent for every business job title
  - Why it fails
    - Semantic failures (hallucinations, misinterpreted instructions, reasoning drift) propagate silently
    - Stronger foundation models do NOT uniformly improve robustness
- Pattern: Coordinate with Clear Roles
  - Role specialization: define agents by capability or domain
  - Explicit coordinator: supervisor agent or routing rules
  - Shared state with strong invariants when needed
  - Termination rules: clear criteria for workflow end
  - Least-privilege tools: each agent gets only what it needs
  - Right-size architecture: start with fewer, broader agents; split only with evidence

13. Responsiveness Afterthought

- Anti-Pattern
  - No upfront latency budget or SLA consideration
  - LLM invoked too often, multiple model calls where one would do
  - Using nested planning loops for latency-sensitive use cases
  - Why it fails
    - 53% of mobile users abandon after 3+ seconds
    - Agent systems with 5–10s response times fail UX requirements
- Pattern: Performance-First Design
  - Set latency budgets upfront: 300–800ms real-time, 1–3s interactive, 5–8s background
  - Minimize planning overhead: invoke planner once per turn
  - Precompute and cache frequent queries
  - Inline deterministic logic as tool or workflow step
  - Parallelize independent tool calls and short-circuit when possible

14. Unbound Execution Cost

- Anti-Pattern
  - Planner always chooses largest model regardless of task complexity
  - No caching of repeated queries or intermediate results
  - Summarizing everything, all the time
  - Why it fails
    - No cost awareness in planner
    - Costs accumulate across small decisions
- Pattern: Make Cost Visible
  - Cost-aware planning: consider cost/benefit, track cost per step and per run
  - Aggressive caching: cache retrieval results and intermediate computations
  - Right-size models: small/fast for routing and classification, large for complex reasoning
  - Minimize summarization: prefer structured state objects over free-text summaries

15. Demo-Grade Agent in Production

- Anti-Pattern
  - Building demo agent with off-the-shelf frameworks, testing on laptop only
  - No concurrency model, no resource management, no observability at scale
  - No multi-user testing, no adversarial testing, no red-teaming
  - Why it fails
    - Hard-coded credentials, file-based repos, no multi-turn handling
    - No failure isolation: one agent's error cascades to others
    - Maintenance burden: team spends time on infrastructure instead of capabilities
- Pattern: Use Production Agent Platforms
  - Leverage mature platforms with solved infrastructure and scale problems
  - Built-in: concurrency, distributed tracing, cost tracking, failure isolation
  - Enterprise infrastructure: scaling, caching, context compression
  - Integration: MCP tools, Agent-to-Agent (A2A) communication
  - Enterprise-ready: auth, compliance, deployment pipelines
  - Build custom only with specific requirements and dedicated team

Knowledge and Document Processing Patterns (16–19)

16. Mixing Extraction with Processing

- Anti-Pattern
  - Cramming extraction AND processing into single Document Extractor node
  - Using verbs like "summarize" or "calculate" in entity definitions
  - Why it fails
    - Document Extractor is prompt-engineered for extraction only
    - Mixing extraction with transformation confuses the model
    - Multi-step logic doesn't fit extraction paradigm
- Pattern: Decompose and Sequence Document Tasks
  - Step 1: Extract raw data exactly as it appears
  - Step 2: Process with appropriate tools (code, prompt nodes, business logic)
  - Step 3: Orchestrate multi-step process (extract → validate → calculate → summarize → return)

17. Lazy Field Definitions

- Anti-Pattern
  - Minimal, vague, or ambiguous field definitions in Document Extractor
  - "amount" (which amount?), "date" (which date?), "address" (which address?)
  - Why it fails
    - Field names/descriptions become part of the extraction prompt
    - Complex documents have multiple similar items needing disambiguation
    - Works for simple documents but fails at scale
- Pattern: Explicit, Unambiguous Field Definitions
  - Start with explicit field names and descriptions
  - Perform error analysis and refine low-quality fields
  - Provide location hints
    - Single page: geo hints (e.g., "top right")
    - Multi-page: section/line number references
  - Add confused fields as additional schema fields to force disambiguation

18. Using RAG for Whole-Document Operations

- Anti-Pattern
  - Using agent knowledge for questions requiring full document or corpus access
  - Expecting RAG to handle counting, aggregation, ordering, or structure operations
  - Why it fails
    - RAG retrieves top matching chunks (5–10), not complete documents
    - Document structure (rows, pages, sections) lost in chunking
    - File type and metadata not preserved in vector embeddings
- Pattern: Use Database or API, Not RAG for Everything
  - Structured data operations: use database + SQL query tool
  - Document structure operations: use appropriate formats (CSV over XLSX)
  - Metadata-based filtering: index metadata separately, use as pre-filter
  - Whole-document analysis: use long-context models or summarization pipelines

19. One-Size-Fits-All Chunking

- Anti-Pattern
  - Using default chunk sizes (512 or 1024 tokens) without understanding content
  - Assuming single chunk size works for all document types
  - Why it fails
    - Different content has different natural boundaries
    - Default sizes are arbitrary, not chosen for your content
    - Content structure (tables, lists, code blocks) needs different strategies
    - Use case requirements vary
- Pattern: Content-Aware Chunking Strategy
  - Analyze content types: different content needs different chunk sizes
  - Preserve semantic boundaries: don't split mid-paragraph or mid-concept
  - Test with real queries: measure retrieval quality
  - Use overlapping chunks for context continuity
  - Tune per use case: FAQ answers, policy documents, technical specs
  - Don't mix multiple knowledge sources into a single knowledge base

Design-Time Patterns (20–25)

20. No Guardrails

- Anti-Pattern
  - No input validation, output filtering, or boundary enforcement
  - Agent accepts any user input and passes it directly to tools or downstream systems
  - No content filtering on model outputs before they reach users or APIs
  - Why it fails
    - Prompt injection: adversarial inputs hijack agent behavior
    - Data leakage: agent exposes sensitive information in responses
    - Tool misuse: malformed or malicious inputs cause unintended tool actions
    - Compliance violations: unfiltered outputs may contain PII, profanity, or off-topic content
    - Cascading trust: downstream systems trust agent output implicitly
- Pattern: Layered Guardrails
  - Input guardrails
    - Schema validation on all user inputs before they reach the LLM
    - Prompt injection detection (classifier or rule-based) as a pre-processing step
    - Input length limits and rate limiting per user/session
    - PII detection and redaction before context injection
  - Output guardrails
    - Content classifiers on model output (toxicity, relevance, factuality)
    - Schema validation on structured outputs before tool calls
    - Blocklists and regex filters for known sensitive patterns
    - Confidence thresholds: low-confidence outputs trigger fallback or human review
  - Tool guardrails
    - Parameter validation at the tool boundary, not just in the prompt
    - Dry-run mode for destructive operations
    - Allowlists for permitted tool-parameter combinations
  - Defence in depth: guardrails at every layer (input → reasoning → output → tool) so no single failure bypasses all protections

21. Implicit Routing

- Anti-Pattern
  - Single agent handles all intents, domains, and complexity levels
  - Routing logic embedded in mega-prompts or long if/else chains
  - No confidence scoring, agent always produces an answer even when unsure
  - Why it fails
    - Accuracy degrades as intent space grows
    - No graceful handling of out-of-domain or ambiguous queries
    - Single prompt cannot optimize for all domains simultaneously
    - Impossible to tune performance per-domain without affecting others
- Pattern: Explicit Routing and Intent Classification
  - Dedicated router
    - Lightweight classifier (small model or embedding-based) as first step
    - Routes to domain-specific agents or workflows based on intent
    - Faster and cheaper than having a large model reason about routing
  - Confidence-based dispatch
    - High confidence → route to specialized agent
    - Medium confidence → route with caveats or ask clarifying question
    - Low confidence → fallback agent, human handoff, or graceful decline
  - Hierarchical routing
    - First stage: broad domain classification (billing, technical, general)
    - Second stage: fine-grained intent within domain
    - Each stage can use different models or methods
  - Routing metadata
    - Log routing decisions, confidence scores, and selected agent
    - Use routing analytics to identify gaps and retrain classifiers
  - Fallback design
    - Always define a default route for unrecognized intents
    - Fallback should acknowledge uncertainty rather than hallucinate

22. No Evaluation Framework

- Anti-Pattern
  - Testing agents manually with a handful of examples
  - No regression suite, no automated eval pipeline
  - Prompt changes deployed without measuring impact
  - Success defined by "it looks right" rather than metrics
  - Why it fails
    - Agents are non-deterministic; manual testing catches a fraction of failure modes
    - Prompt changes have unpredictable downstream effects
    - No baseline to detect degradation over time
    - Cannot compare agent versions objectively
    - Production issues discovered by users, not by tests
- Pattern: Continuous Evaluation Pipeline
  - Eval dataset design
    - Golden datasets with expected outputs per task type
    - Include adversarial, edge-case, and failure-mode examples
    - Segment by difficulty, domain, and intent type
  - Metric layers
    - Task-level: accuracy, tool selection correctness, goal completion rate
    - Step-level: per-step correctness, hallucination rate, tool call success
    - System-level: latency (p50, p95, p99), cost per task, error rate
    - User-level: satisfaction, escalation rate, retry rate
  - Regression testing
    - Run eval suite on every prompt, tool, or model change
    - Compare against baseline with statistical significance
    - Block deployments that regress on critical metrics
  - Online evaluation
    - A/B testing for prompt and model changes in production
    - Shadow mode: run new version alongside old, compare outputs without serving
    - Canary deployments: gradual rollout with metric monitoring
  - Red-teaming
    - Periodic adversarial testing by dedicated team
    - Prompt injection attempts, role confusion, boundary testing
    - Document findings and add to eval dataset

23. Prompt Fragility

- Anti-Pattern
  - Prompts hardcoded as inline strings scattered across codebase
  - No version control, no templating, no separation of concerns
  - Prompt changes require code deployments
  - No way to trace which prompt version produced which output
  - Why it fails
    - Prompts are the most frequently changed component but treated as static code
    - No audit trail for prompt changes
    - Cannot roll back a bad prompt change independently of code
    - Multiple developers editing prompts creates merge conflicts and inconsistencies
    - Testing prompt changes requires full deployment cycle
- Pattern: Prompt as a Managed Artifact
  - Version control
    - Prompts stored as separate files (YAML, JSON, or Markdown) in version control
    - Each prompt has a version identifier and changelog
    - Prompt changes reviewed like code changes (PR review)
  - Templating
    - Parameterized prompts with clear variable boundaries
    - Shared components (persona, output format, guardrail instructions) as reusable partials
    - Environment-specific overrides (dev, staging, prod)
  - Prompt registry
    - Central catalog of all prompts with metadata (owner, last modified, linked eval results)
    - API-driven prompt retrieval so changes don't require code redeployment
    - Rollback capability: revert to previous version instantly
  - Observability
    - Log prompt version with every LLM call
    - Correlate prompt versions with eval metrics and production errors
    - Alert on metric degradation tied to prompt changes

24. Bolted-On Human-in-the-Loop

- Anti-Pattern
  - HITL added as afterthought when agent fails, not designed into architecture
  - No clear escalation criteria; humans called in ad-hoc
  - No feedback path from human corrections back into the system
  - Human overrides not logged or used for improvement
  - Why it fails
    - Unclear escalation criteria means either too many or too few escalations
    - No SLA on human response creates unpredictable latency
    - Human corrections are wasted if not captured as training signal
    - Agents never improve on tasks they consistently escalate
    - User experience degrades at handoff boundaries
- Pattern: Designed Human-Agent Collaboration
  - Escalation design
    - Define explicit triggers: confidence thresholds, risk levels, domain boundaries, tool failures
    - Categorize escalations: approval-required, review-required, exception-handling
    - Set SLAs per escalation type (30s for approvals, 4h for reviews)
  - Handoff UX
    - Preserve full context when handing to human (don't make them re-read the thread)
    - Structured handoff: summary of what agent did, what it's unsure about, proposed next steps
    - Allow human to approve, modify, or reject agent's proposed action
  - Feedback loops
    - Log every human correction with the original agent output
    - Use corrections as fine-tuning or few-shot examples
    - Track escalation rate per task type; decreasing rate indicates agent improvement
  - Progressive autonomy
    - Start with human-in-the-loop for new task types
    - Graduate to human-on-the-loop (notification only) as confidence builds
    - Move to full autonomy only with eval evidence and stakeholder sign-off

25. Stateless Between Sessions

- Anti-Pattern
  - Agent has no memory beyond current conversation
  - Every session starts from scratch regardless of prior interactions
  - User preferences, past decisions, and context lost between sessions
  - Why it fails
    - Users repeat themselves across sessions
    - Agent cannot learn from prior corrections or preferences
    - Personalization impossible without memory
    - Multi-session workflows (e.g., onboarding) cannot maintain continuity
    - No ability to reference prior decisions or commitments
- Pattern: Structured Memory Architecture
  - Memory types
    - Working memory: current session context (conversation + state)
    - Short-term memory: recent sessions, recent tool outputs, recent decisions (hours/days)
    - Long-term memory: user preferences, learned patterns, domain knowledge (persistent)
    - Episodic memory: specific past interactions retrievable by similarity or recency
  - Storage design
    - Working memory: in-context or external state object (see Pattern #3)
    - Short-term: key-value store with TTL, session-scoped
    - Long-term: vector store or structured DB with user-scoped access controls
    - Episodic: indexed conversation summaries with metadata (date, topic, outcome)
  - Retrieval strategy
    - Selective injection: retrieve only relevant memories, not entire history
    - Relevance scoring: rank memories by similarity to current query
    - Recency weighting: more recent memories ranked higher by default
    - Memory budget: fixed token allocation for memory context
  - Memory hygiene
    - Expiration policies: TTL on short-term, periodic review on long-term
    - Contradiction resolution: newer information supersedes older
    - User control: users can view, correct, or delete their stored memories
    - Privacy: memory scoped per user, encrypted at rest, access-controlled

Operational Patterns (26–31)

26. Flying Blind

- Anti-Pattern
  - No structured logging, tracing, or monitoring of agent behavior
  - Debugging production issues by reading raw conversation logs
  - No visibility into tool call success rates, latency, or cost breakdown
  - Why it fails
    - Cannot diagnose why agent made a specific decision
    - No alerting on degradation; issues discovered by user complaints
    - Cannot attribute cost or latency to specific steps
    - Root cause analysis takes hours instead of minutes
    - No data to drive optimization decisions
- Pattern: End-to-End Observability
  - Structured tracing
    - Unique trace ID per request spanning all agent steps
    - Each step logged: input, output, model used, tokens consumed, latency, tool called
    - Parent-child relationships for multi-agent or workflow handoffs
  - Dashboards
    - Real-time: active sessions, error rate, p95 latency, cost per minute
    - Historical: trends in accuracy, escalation rate, cost per task, tool usage
    - Per-agent: breakdown by agent role showing individual performance
  - Alerting
    - Error rate spikes, latency threshold breaches, cost anomalies
    - Tool failure rate changes, escalation rate increases
    - Model output quality drops (if online eval is running)
  - Replay and debugging
    - Full trace replay: step through any request to see exactly what happened
    - Counterfactual: re-run a trace with different prompts or models
    - Diff mode: compare traces from two agent versions side by side

27. No Caching Strategy

- Anti-Pattern
  - Every request hits the LLM regardless of whether the answer was computed before
  - Repeated tool calls for the same data within the same session
  - No distinction between volatile and stable data in retrieval pipelines
  - Why it fails
    - Identical queries pay full LLM cost and latency every time
    - Tool APIs hit rate limits or charge per call unnecessarily
    - Retrieval of static reference data repeated on every turn
    - Cost and latency scale linearly with request volume instead of sublinearly
- Pattern: Multi-Layer Caching
  - Semantic cache
    - Cache LLM responses keyed by semantic similarity of input (not exact match)
    - Embedding-based lookup with similarity threshold
    - TTL based on content volatility (short for real-time data, long for reference)
  - Tool result cache
    - Cache tool outputs with appropriate TTL per tool
    - Static reference data (product catalog, policy documents): long TTL
    - Dynamic data (account balance, inventory): short TTL or no cache
    - Cache key includes tool name + normalized parameters
  - Retrieval cache
    - Cache vector search results for repeated or similar queries
    - Invalidate on index update
  - Context cache
    - Reuse computed context (system prompt + tools + few-shot) across turns
    - Provider-level prompt caching (where available) to reduce prefill cost
  - Cache invalidation
    - Event-driven invalidation for data changes
    - TTL-based expiration as fallback
    - Manual purge capability for emergency corrections

28. Context Window Mismanagement

- Anti-Pattern
  - Stuffing everything into context and hoping the model sorts it out
  - No strategy for what enters context window or in what order
  - Conversation history grows unbounded until context limit hit
  - Why it fails
    - Lost in the Middle: information in the middle of long context is poorly attended to
    - Context overflow causes silent truncation of critical information
    - Older but important context displaced by recent but irrelevant content
    - Token cost grows linearly with context size
    - Model reasoning quality degrades as context fills with noise
- Pattern: Context Window Engineering
  - Priority-based allocation
    - System prompt and guardrails: always present, highest priority
    - Current task state: always present
    - Relevant tools: only tools applicable to current step
    - Retrieved context: ranked by relevance, truncated to budget
    - Conversation history: summarized or windowed, lowest priority
  - Conversation management
    - Sliding window: keep last N turns verbatim, summarize older turns
    - Compressive summarization: periodically compress history into structured summary
    - Selective retention: keep turns with decisions, tool outputs, or user corrections
  - Token budgeting
    - Allocate fixed token budgets per context section
    - Monitor actual usage and alert on budget overruns
    - Adjust budgets based on task type (complex reasoning needs more working space)
  - Placement strategy
    - Critical information at beginning and end of context (primacy and recency effects)
    - Tool definitions grouped and ordered by likely relevance
    - Few-shot examples placed immediately before the task

29. RAG as a Black Box

- Anti-Pattern
  - Default RAG pipeline (embed → store → retrieve → generate) with no tuning
  - No query preprocessing, no reranking, no retrieval evaluation
  - Assuming embedding similarity equals answer quality
  - Why it fails
    - User queries are often vague, misspelled, or use different terminology than source docs
    - Embedding similarity is a weak proxy for answer relevance
    - Top-K retrieval without reranking returns quantity over quality
    - No feedback loop to identify and fix retrieval failures
    - Hybrid queries (keyword + semantic) outperform pure semantic in most benchmarks
- Pattern: Retrieval-Augmented Generation Engineering
  - Query preprocessing
    - Query rewriting: LLM reformulates vague queries into precise search queries
    - Query expansion: add synonyms, related terms, acronyms
    - Query decomposition: split complex questions into sub-queries
    - HyDE (Hypothetical Document Embeddings): generate hypothetical answer, embed that instead
  - Retrieval strategy
    - Hybrid search: combine dense (embedding) + sparse (BM25/keyword) retrieval
    - Multi-index: separate indexes for different content types or sources
    - Metadata pre-filtering: filter by date, source, document type before semantic search
    - Recursive retrieval: retrieve, assess, refine query, retrieve again
  - Reranking
    - Cross-encoder reranker on top-K results before injection into context
    - Diversity reranking: ensure results cover different aspects of the query
    - Relevance threshold: discard results below minimum score rather than always using top-K
  - Evaluation
    - Retrieval metrics: precision@K, recall@K, MRR, NDCG
    - End-to-end metrics: answer correctness, faithfulness, relevance
    - Failure analysis: log queries with poor retrieval for iterative improvement
    - Golden dataset: curated query-document pairs for regression testing

30. No Feedback Loop

- Anti-Pattern
  - Agent ships and never improves from real-world usage
  - User corrections, thumbs-up/down, escalations not captured or used
  - No drift detection; model or data changes degrade quality silently
  - Why it fails
    - Agent accuracy slowly degrades as world knowledge changes
    - Same mistakes repeated because corrections never feed back
    - No signal to prioritize which capabilities to improve
    - Cannot detect when model updates or data changes cause regression
    - Missed opportunity to build domain-specific training data
- Pattern: Continuous Improvement Loop
  - Signal collection
    - Explicit feedback: thumbs up/down, corrections, ratings
    - Implicit feedback: retries, rephrasing, escalations, task abandonment
    - Human-in-the-loop corrections (see Pattern #24)
    - Tool success/failure rates per task type
  - Analysis pipeline
    - Aggregate feedback by task type, agent, intent category
    - Identify systematic failure patterns (not just individual errors)
    - Cluster similar failures for root cause analysis
    - Track metrics over time windows to detect drift
  - Improvement actions
    - Prompt refinement: use failure examples to improve instructions
    - Few-shot updates: add corrected examples to prompt
    - Tool improvements: fix tools that consistently fail or return poor data
    - Eval dataset expansion: add real-world failures to regression suite
    - Routing updates: reclassify intents that consistently misroute
  - Drift detection
    - Monitor key metrics on rolling windows (daily, weekly)
    - Alert on statistically significant degradation
    - Automated comparison against baseline after model or data updates
    - Scheduled re-evaluation against golden datasets

31. Security as an Afterthought

- Anti-Pattern
  - Agent has broad permissions to all tools, data sources, and APIs
  - No audit trail of what the agent accessed or modified
  - Secrets (API keys, tokens) embedded in prompts or tool configs
  - No threat modeling for agentic-specific attack vectors
  - Why it fails
    - Compromised agent has blast radius of all connected systems
    - Prompt injection can weaponize agent's own tool access
    - No forensic capability after a security incident
    - Secrets in prompts leak through logging, caching, or model providers
    - Traditional security models don't account for non-deterministic AI actors
- Pattern: Security by Design for Agents
  - Least-privilege access
    - Each agent gets minimum permissions required for its role
    - Scoped API keys per agent, per environment
    - Read-only access by default; write access requires explicit grant
    - Tool-level authorization: agent can call search but not delete
  - Secrets management
    - Secrets injected at runtime, never in prompts or tool definitions
    - Rotate credentials on schedule, revoke on agent decommission
    - Secrets never logged, cached, or included in traces
  - Audit trail
    - Log every tool call with parameters, caller identity, timestamp, outcome
    - Log data accessed (not data content) for compliance
    - Immutable audit logs, tamper-evident storage
    - Retention policies aligned with compliance requirements
  - Threat modeling
    - Prompt injection: test with known attack patterns, add input guardrails
    - Tool abuse: rate limiting, parameter validation, destructive-action gating
    - Data exfiltration: monitor for unusual data access patterns
    - Privilege escalation: ensure agent cannot modify its own permissions
    - Supply chain: audit third-party tools and MCP servers (see Pattern #10)
  - Incident response
    - Kill switch: ability to disable any agent immediately
    - Blast radius containment: agents isolated so one compromise doesn't cascade
    - Forensic replay: reconstruct full trace of compromised session

Quick Decision Guide

- Tool vs Agent vs Workflow
  - Is there only one reasonable action? → TOOL
  - Can steps be known and fixed? → WORKFLOW
  - Steps are dynamic and unpredictable? → AGENT
- Refinements
  - Verbatim output required → Bypass LLM in Tool response or Flow HITL
  - Strict order required → Workflow
  - Open-ended/ambiguous → Agent with smallest tool set

Key Principles

- Architecture wins at scale, not prompt engineering
- Agents reason; workflows execute
- Autonomy must be calibrated, not all-or-nothing
- Tools should filter, not firehose
- External components are dependencies: verify before trusting
- Observability is non-negotiable
- Cost and latency are design constraints, not afterthoughts